EduGenie

Google Gemini-Powered Learning Assistant

Vetri Thiran Training Programme

Topic: Backend API with FastAPI

Project Documentation

English Version

1. Introduction

EduGenie is a Google Gemini-powered learning assistant designed to help students learn through interactive questions, explanations, examples, summaries, and practice activities. The backend API provides the server-side layer that receives requests from the user interface, processes learning requests, communicates with the Gemini AI service, and returns structured responses.

FastAPI is used to build the backend because it supports modern Python development, automatic API documentation, request validation, and asynchronous programming.

2. Objectives

- Build a REST API for the EduGenie learning assistant.
- Accept student questions and learning requests through HTTP endpoints.
- Connect the backend with a Google Gemini model/API.
- Return clear and structured AI-generated learning responses.
- Validate incoming data using Pydantic models.
- Provide automatic interactive API documentation using FastAPI.
- Create a foundation that can later be connected to a web or mobile frontend.

3. Proposed Backend Architecture

Layer	Responsibility
Frontend / Client	Sends student questions and receives learning responses.
FastAPI Backend	Defines endpoints, validates requests, handles application logic.
Service Layer	Builds prompts and manages communication with the Gemini service.
Gemini AI	Generates explanations, examples, summaries, and practice content.
Database (Optional)	Stores users, topics, learning history, feedback, and progress.

4. Technology Stack

- **Python:** Main programming language.
- **FastAPI:** Backend web framework for REST APIs.
- **Pydantic:** Request and response data validation.
- **Uvicorn:** ASGI server used to run the FastAPI application.
- **Google Gemini:** Generative AI component for learning assistance.
- **JSON:** Data format exchanged between client and API.
- **Database (optional):** SQLite, PostgreSQL, or another suitable database for persistent data.

5. API Modules / Endpoints

Method	Endpoint	Purpose
GET	/health	Check whether the backend service is running.
POST	/ask	Send a learning question to the AI assistant.
POST	/explain	Request an explanation of a selected topic.
POST	/quiz	Generate practice questions for a topic.
POST	/summary	Generate a concise topic summary.
POST	/feedback	Submit feedback about an AI response.

6. Project Folder Structure

```
edugenie/
■■■ app/
■   ■■■ main.py
■   ■■■ models.py
■   ■■■ routes/
■   ■   ■■■ learning.py
■   ■   ■■■ health.py
■   ■■■ services/
■   ■   ■■■ gemini_service.py
■   ■■■ config.py
■■■ requirements.txt
■■■ README.md
```

7. FastAPI Basic Implementation

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="EduGenie API")

class Question(BaseModel):
    topic: str
    question: str

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/ask")
def ask(data: Question):
    # Send data.question to the Gemini service
    # and return the generated learning response.
    return {
        "topic": data.topic,
        "answer": "Gemini-generated learning response"
    }
```

8. Gemini Service Integration

The Gemini integration should be kept in a separate service module instead of placing all AI logic directly inside the API route. The backend receives the student's request, constructs a safe and focused prompt, sends it to the configured Gemini service, and returns the generated content to the client.

```
async def generate_learning_response(topic, question):
    prompt = f"""
    You are EduGenie, a learning assistant.
    Explain the following topic clearly for a student.
    Topic: {topic}
    Question: {question}
```

```
    """
    # Call the configured Gemini API/client here.
    # Return the model response.
    return response_text
```

9. Request and Response Example

POST /ask – Request

```
{
  "topic": "Python",
  "question": "What is a Python function?"
}
```

Response

```
{
  "topic": "Python",
  "answer": "A function is a reusable block of Python code..."
}
```

10. Backend Workflow

Step	Action
1	Student enters a topic and question in the EduGenie frontend.
2	Frontend sends an HTTP request to the FastAPI /ask endpoint.
3	Pydantic validates the request data.
4	FastAPI passes the request to the Gemini service layer.
5	The service creates a learning-focused prompt.
6	Gemini generates the response.
7	FastAPI returns the response as JSON.
8	Frontend displays the answer to the student.

11. Installation and Run Commands

```
# Create a virtual environment
python -m venv venv

# Activate the environment
# Windows:
venv\Scripts\activate

# Install dependencies
pip install fastapi uvicorn pydantic

# Run the API
uvicorn app.main:app --reload

# Open interactive API documentation
http://127.0.0.1:8000/docs
```

12. API Documentation

FastAPI automatically provides interactive documentation. The Swagger UI is normally available at /docs, while an alternative ReDoc interface is available at /redoc. These interfaces allow developers to inspect endpoints, request models, and test API calls during development.

13. Security and Best Practices

- Keep Gemini API credentials in environment variables or a secure secret-management system.
- Do not hard-code API keys in source code or publish them in repositories.
- Validate and sanitize incoming user input.
- Use HTTPS when deploying the API publicly.
- Add authentication and authorization when the application contains user accounts or private data.
- Apply rate limits and error handling to protect the service.
- Avoid sending unnecessary personal information to an AI service.

14. Testing

- Test the /health endpoint to verify service availability.
- Test valid and invalid JSON requests.

- Test missing or incorrect fields using Pydantic validation.
- Test Gemini service failures and timeout handling.
- Check that API responses follow the expected JSON structure.
- Perform integration testing between the frontend, FastAPI backend, and Gemini service.

15. Expected Outcome

The completed backend provides a clean API layer for EduGenie. It enables a frontend application to request AI-powered educational assistance without directly managing the AI integration. The modular design also makes it easier to add new learning features such as quizzes, summaries, progress tracking, and feedback.

16. Conclusion

The EduGenie FastAPI backend demonstrates how Python, FastAPI, Pydantic, and Google Gemini can be combined to create an AI-powered learning assistant. The API-based architecture separates the frontend from the AI service, provides structured endpoints, and creates a scalable foundation for an educational application under the Vetri Thiran Training Programme.

Keywords: EduGenie, Google Gemini, FastAPI, Backend API, REST API, Python, Generative AI, Learning Assistant, Vetri Thiran Training Programme